

Backend Fundamentals — Recap 4

Real database persistence, real bugs, and shipping it to GitHub

Milestone: the in-memory array is gone. Every route now reads and writes real, persistent data, and the project is live on GitHub.

1. Why SQLite

Unlike Postgres or MySQL, SQLite needs no separate server process — the entire database is a single file on disk (e.g. `tasks.db`). This is what actually fixes the earlier problem of tasks disappearing on restart: the array lived in RAM and reset every time the process restarted; a file on disk does not.

```
const Database = require('better-sqlite3');
const db = new Database('tasks.db');
```

```
db.exec(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL
  )
`);
```

AUTOINCREMENT means the database itself now assigns unique ids — replacing the manual `nextId` counter entirely.

2. Running Queries: `.all()` / `.get()` / `.run()`

```
db.prepare('SELECT * FROM tasks').all();           // many rows
db.prepare('SELECT * FROM tasks WHERE id = ?').get(id); // one row
db.prepare('INSERT INTO tasks (title) VALUES (?)').run(title); // write action
```

`.all()` for many rows, `.get()` for one row, `.run()` for INSERT/UPDATE/DELETE (actions that don't return rows).

3. The ? Placeholder — Security, Not Style

Values from the user are never inserted directly into the SQL string. A `?` marks where a value goes, and it's supplied separately as an argument to `.run()`/`.get()`/`.all()` — in the same order they appear. This is what prevents **SQL injection**, a real attack where malicious input is crafted to run as code instead of being treated as a plain value.

```
db.prepare('UPDATE tasks SET title = ? WHERE id = ?').run(newTitle, id);
// two ? marks -> two arguments, in order
```

4. Getting the New Row Back After Insert

```
const result = db.prepare('INSERT INTO tasks (title) VALUES (?)').run(title);
const newTask = db.prepare('SELECT * FROM tasks WHERE id = ?').get(result.lastInsertRowid);
```

`result.lastInsertRowid` holds the id SQLite just auto-assigned — used immediately to fetch the full new row back for the response.

5. Real Bugs Fixed Along the Way

Missing parentheses:

```
db.prepare('SELECT * FROM tasks').all;    // wrong: refers to the function, never calls it
db.prepare('SELECT * FROM tasks').all();  // correct: calls it, returns the rows
```

The error `Received function all` was the direct clue — `res.send()` was handed a function instead of data.

Wrong data source in PUT:

```
db.prepare(...).run(req.params.title, req.params.id) // wrong: no :title in the URL
db.prepare(...).run(req.body.title, req.params.id)   // correct: title comes from the body
```

A route also silently returned nothing at all when its `res.send()` was left commented out — a reminder that every code path needs to actually send a response.

6. Why `Number(req.params.id)` Is No Longer Needed

SQLite columns have type affinity — an `INTEGER` column automatically converts a comparison value like the text `"1"` into a number before comparing. JavaScript's strict `===` required matching types by hand; SQL's `WHERE id = ?` does that conversion for you.

7. Proof: Data Survives a Restart

Tasks created via `POST`, confirmed via `GET`, then the server fully stopped and restarted — the same tasks were still there. This is the actual fix for the original problem: an in-memory array resets on restart; a database file on disk does not.

8. Cleanup: Removing Dead Code

Once every route used `db`, the old `let tasks = []` and `let nextId = 1` were unused — left-over clutter from before the migration, removed once no route depended on them anymore.

9. What "REST" Actually Stands For

REpresentational **S**tate **T**ransfer — a set of design ideas, not a technology. A **resource** is a thing (a task); a **representation** is the JSON sent to describe its current state; **state transfer** means each response hands back the resource's current state. A REST API is also **stateless** — each request is self-contained, and the server doesn't remember previous requests between them.

10. Shipping It: Git and GitHub

.gitignore excludes files that shouldn't be tracked: `node_modules/` (regenerable via `npm install`) and `tasks.db` (local test data, auto-recreated by the code itself).

```
git init
git add .
git commit -m "Task API: CRUD, validation, error handling, SQLite persistence"
git branch -M main
git remote add origin <repo-url>
git push -u origin main
```

A README was written describing the project, its features, tech stack, and API reference — what makes a repo readable to someone who isn't the author.

11. Where the API Stands Now

- Fully migrated from an in-memory array to persistent SQLite storage
- All five CRUD routes read/write through the database, with correct status codes
- Validation and error handling carried over intact from before
- Pushed to GitHub with a `.gitignore` and a professional README

Next up: idempotency keys — making POST safe to retry without creating duplicates.